# RmlUI

## Starter Guide

for RecoilEngine game developers

*Build reactive, HTML/CSS-style game UIs in Lua*

| | |
|---|---|
| Engine | RecoilEngine (Spring) |
| Framework | RmlUi (mikke89/RmlUi) |
| Language | Lua (via sol2 bindings) |
| Availability | LuaUI (LuaIntro / LuaMenu planned) |

# Table of Contents

# 1. Introduction

RmlUI is a UI framework that lets you build reactive game interfaces using an HTML/CSS-style workflow — with Lua instead of JavaScript. If you have ever built a web page, the learning curve is gentle. If you haven't, the concepts are still approachable because the mental model (mark-up + style + logic) is widely documented.

RecoilEngine ships with a full RmlUI integration including:

- An OpenGL 3 renderer tightly coupled to the engine's rendering pipeline
- A virtual-filesystem-aware file loader so your .rml and .rcss files live inside game archives
- Complete Lua bindings via sol2 so every RmlUI object is accessible from Lua
- Custom elements: **<texture>** for engine textures and **<svg>** for vector art
- A built-in DOM debugger for interactive inspection

This guide walks you through everything you need to get a working, reactive widget running in LuaUI. The examples are game-engine-agnostic and safe to copy into any Recoil-based game.

> ■ **NOTE**
>
> RmlUI is currently available in **LuaUI** (the in-game UI). Support for LuaIntro and LuaMenu is planned for a future release.

# 2. Key Concepts

Before writing any code it helps to understand the five core objects:

| Concept | Description |
|---|---|
| Context | A named container that holds documents and data models. Multiple contexts can exist simultaneously, each rendered independently. |
| Document | A parsed RML file representing a DOM tree. Documents live inside a Context and can be shown/hidden/closed. |
| RML | The markup language for documents. Very similar to XHTML (well-formed XML). The root tag is <rml> instead of <html>. |
| RCSS | The styling language. Similar to CSS2 with some differences (see section 11). File extension: .rcss. |
| Data Model | A Lua table exposed to the RML document via reactive data bindings. Changes to the model automatically update the rendered UI. |

On the **Lua side** you create contexts, attach data models to them, and load documents. On the **RML side** you declare your UI structure and reference data-model values through data bindings. Each widget will typically consist of at least three files: a .lua, a .rml, and optionally a .rcss file — grouping them in a folder per widget keeps things tidy.

# 3. Setup

RecoilEngine ships a minimal setup script at `cont/LuaUI/rml_setup.lua`. It is included automatically by the base content handler but you should understand what it does so you can extend it for your game.

## 3.1 The rml_setup.lua script

The script performs three tasks: guards against double-initialisation, loads font faces, and registers mouse-cursor aliases. Below is a more complete version, as used by Beyond All Reason, that also creates a shared context and configures dp scaling:

```
--  luaui/rml_setup.lua
--  author:  lov + ChrisFloofyKitsune
--  License: GNU GPL, v2 or later

if (RmlGuard or not RmlUi) then
    return
end
-- Prevent this from running more than once
RmlGuard = true

-- Patch CreateContext to set dp_ratio automatically
local oldCreateContext = RmlUi.CreateContext
local function NewCreateContext(name)
    local context = oldCreateContext(name)
    local viewSizeX, viewSizeY = Spring.GetViewGeometry()
    local userScale = Spring.GetConfigFloat("ui_scale", 1)
    local baseWidth, baseHeight = 1920, 1080
    local resFactor = math.min(viewSizeX / baseWidth, viewSizeY / baseHeight)
    -- Floor to 2 decimal places to avoid floating point drift
    context.dp_ratio = math.floor(resFactor * userScale * 100) / 100
    return context
end
RmlUi.CreateContext = NewCreateContext

-- Load fonts (list your .ttf files here)
local font_files = {
    -- "Fonts/MyFont-Regular.ttf",
}
for _, file in ipairs(font_files) do
    RmlUi.LoadFontFace(file, true)
end

-- Map CSS cursor names to engine cursor names
-- CSS cursor list: https://developer.mozilla.org/en-US/docs/Web/CSS/cursor
RmlUi.SetMouseCursorAlias("default", 'cursornormal')
RmlUi.SetMouseCursorAlias("move",    'uimove')
RmlUi.SetMouseCursorAlias("pointer", 'Move')

-- Create the shared context used by all widgets
RmlUi.CreateContext("shared")
```

## 3.2 Including the setup in main.lua

Add a single line to `luaui/main.lua` to run the setup before any widgets load:

```
-- luaui/main.lua
VFS.Include(LUAUI_DIRNAME .. "rml_setup.lua", nil, VFS.ZIP)
```

**■ NOTE**

> The base-content rml_setup.lua only loads a font and sets cursor aliases; it does **not** create a context. If you are building a new game you should add context creation here or let each widget create its own context.

# 4. Writing Your First Document (.rml)

Create a file at `luaui/widgets/my_widget/my_widget.rml`. RML is well-formed XML, so every tag must be closed and attribute values must be quoted.

## 4.1 Root structure

```
<rml>
<head>
    <title>My Widget</title>
    <!-- External stylesheet (optional) -->
    <link type="text/rcss" href="my_widget.rcss"/>
    <!-- Inline styles -->
    <style>
        body { margin: 0; padding: 0; }
    </style>
</head>
<body>
    <!-- Your content here -->
</body>
</rml>
```

## 4.2 Styles (inline and .rcss)

RmlUI starts with **no default styles** at all — not even block/inline defaults. The RmlUI docs provide an HTML4 base stylesheet you can copy in, but you will need to add form element styles yourself. Use `dp` units instead of `px` so your UI scales correctly with the player's display and `ui_scale` setting.

```
/* Typical widget container */
#my-widget {
    position: absolute;
    width: 400dp;
    right: 10dp;
    top: 50%;
    transform: translateY(-50%);
    pointer-events: auto;   /* required to receive mouse input */
}

#my-widget .panel {
    padding: 10dp;
    border-radius: 8dp;
    border: 1dp #6c7086;   /* note: no 'solid' keyword – see section 11 */
    background-color: rgba(30, 30, 46, 200);
}
```

## 4.3 Data bindings in RML

Attach a data model to a root element with `data-model="model_name"`. All data binding attributes inside that element can reference the model.

```
<body>
  <!-- data-model scopes all bindings inside this div -->
  <div id="my-widget" data-model="my_model">

    <!-- Text interpolation -->
    <p>Status: {{status_message}}</p>

    <!-- Conditional visibility -->
    <div data-if="show_details">
        <p>Extra details go here.</p>
    </div>

    <!-- Dynamic class based on model value -->
    <div class="panel" data-class-active="is_active">
        Active panel
    </div>

    <!-- Loop over an array -->
    <ul>
        <li data-for="item, i: items">{{i}}: {{item.label}}</li>
    </ul>

    <!-- Two-way checkbox binding -->
    <input type="checkbox" data-checked="is_enabled"/>

    <!-- Data event: calls model function on click -->
    <button data-click="on_button_click()">Click me</button>

  </div>
</body>
```

| Binding | Effect |
|---|---|
| {{value}} | Interpolate model value as text |
| data-model="name" | Attach named data model to this element and its children |
| data-if="flag" | Show element only when flag is truthy |
| data-class-X="flag" | Add class X when flag is truthy |
| data-for="v, i: arr" | Repeat element for each item in array arr |
| data-checked="val" | Two-way bind checkbox to boolean model value |
| data-click="fn()" | Call model function on click (data event) |
| data-attr-src="val" | Dynamically set the src attribute from model value |

# 5. The Lua Widget

Each widget is a Lua file that the handler loads. The `widget` global is injected by the widget handler and provides the lifecycle hooks.

## 5.1 Widget skeleton

```
-- luaui/widgets/my_widget/my_widget.lua
if not RmlUi then return end   -- guard: RmlUi not available

local widget = widget ---@type Widget

function widget:GetInfo()
    return {
        name    = "My Widget",
        desc    = "Demonstrates RmlUI basics.",
        author  = "YourName",
        date    = "2025",
        license = "GNU GPL, v2 or later",
        layer   = 0,
        enabled = true,
    }
end
```

## 5.2 Loading the document

```
local RML_FILE = "luaui/widgets/my_widget/my_widget.rml"
local MODEL_NAME = "my_model"

local rml_ctx    -- the RmlUi Context
local dm_handle  -- the DataModel handle (MUST be kept – see section 6)
local document   -- the loaded Document

-- Initial data model state
local init_model = {
    status_message = "Ready",
    is_enabled     = false,
    show_details   = false,
    items = {
        { label = "Alpha", value = 1 },
        { label = "Beta",  value = 2 },
    },
    -- Functions can live in the model too (callable from data-events)
    on_button_click = function()
        Spring.Echo("Button was clicked!")
    end,
}

function widget:Initialize()
    rml_ctx = RmlUi.GetContext("shared")

    -- IMPORTANT: assign dm_handle or the engine WILL crash on RML render
    dm_handle = rml_ctx:OpenDataModel(MODEL_NAME, init_model)
    if not dm_handle then
        Spring.Echo("[my_widget] Failed to open data model")
        return
    end

    document = rml_ctx:LoadDocument(RML_FILE, widget)
    if not document then
        Spring.Echo("[my_widget] Failed to load document")
        return
    end

    document:ReloadStyleSheet()
    document:Show()
end
```

## 5.3 Shutting down cleanly

```
function widget:Shutdown()
    if document then
        document:Close()
        document = nil
    end
    -- Remove the model from the context; frees memory
    rml_ctx:RemoveDataModel(MODEL_NAME)
    dm_handle = nil
end
```

# 6. Data Models in Depth

## 6.1 Opening a data model

`context:OpenDataModel(name, table)` copies the structure of the table into a reactive proxy. The name must match the `data-model="name"` attribute in your RML. It must be unique within the context.

> ■ **WARNING**
>
> You **must** store the return value of OpenDataModel in a variable. Letting it be garbage-collected while the document is open will crash the engine the next time RmlUI tries to render a data binding.

## 6.2 Reading and writing values

For simple string/number/boolean values at the top level of the model you can use dot-notation directly on the handle:

```
-- Read a value
local msg = dm_handle.status_message   -- works

-- Write a value (auto-marks the field dirty; UI updates next frame)
dm_handle.status_message = "Unit selected"
dm_handle.is_enabled     = true
```

## 6.3 Iterating arrays

Because the handle is a sol2 proxy, you cannot iterate it directly with `pairs` or `ipairs`. Call `dm_handle:__GetTable()` to get the underlying Lua table:

```
local tbl = dm_handle:__GetTable()

for i, item in ipairs(tbl.items) do
    Spring.Echo(i, item.label, item.value)
    item.value = item.value + 1   -- modify in place
end
```

### 6.4 Marking dirty

When you modify nested values (e.g. elements inside an array) through `__GetTable()`, you must tell the model which top-level key changed so that RmlUI re-renders the bound elements:

```
-- After modifying tbl.items …
dm_handle:__SetDirty("items")

-- For a simple top-level assignment via dm_handle.key = value,
-- dirty-marking is done automatically.
```

## 7. Events

Recoil's RmlUI supports two distinct event families. They look similar but have different scopes — mixing them up is a common source of confusion.

### 7.1 Normal events (on*)

Written as `onclick="myFunc()"`. The function is resolved from the **second argument** passed to `LoadDocument` — the 'widget' table. These events do **not** have access to the data model.

```
-- Lua
local doc_scope = {
    greet = function(name)
        Spring.Echo("Hello", name)
    end,
}
document = rml_ctx:LoadDocument(RML_FILE, doc_scope)
```

```
<!-- RML -->
<button onclick="greet('World')">Say hello</button>
```

### 7.2 Data events (data-*)

Written as `data-click="fn()"`. The function is resolved from the **data model**. These events have full access to all model values.

```
-- Lua model
local init_model = {
    counter = 0,
    increment = function()
        -- dm_handle is captured in the closure
        dm_handle.counter = dm_handle.counter + 1
    end,
}
```

```
<!-- RML -->
<p>Count: {{counter}}</p>
<button data-click="increment()">+1</button>
```

| Type | Syntax example | Scope |
|---|---|---|
| Normal | onclick="fn()" | LoadDocument second argument (widget table) |
| Data | data-click="fn()" | Data model table |
| Normal | onmouseover="fn()" | LoadDocument second argument |
| Data | data-mouseover="fn()" | Data model table |

# 8. Custom Recoil Elements

Beyond standard HTML elements, Recoil adds two custom RML elements.

## 8.1 The <texture> element

Renders any engine texture — unit icons, map thumbnails, GL4 render targets, etc. Behaves identically to `<img>` except the `src` attribute takes a **Recoil texture reference string** (see engine docs for the full format).

```
<!-- Load a file from VFS -->
<texture src="unitpics/armcom.png" width="64dp" height="64dp"/>

<!-- Reference a named GL texture -->
<texture src="%luaui:myTextureName" width="128dp" height="128dp"/>
```

## 8.2 The <svg> element

Renders an SVG image. Unlike upstream RmlUI, the Recoil implementation accepts either a file path **or raw inline SVG data** in the `src` attribute:

```
<!-- External file -->
<svg src="images/icon.svg" width="32dp" height="32dp"/>

<!-- Inline SVG data -->
<svg src="<svg xmlns='http://www.w3.org/2000/svg' viewBox='0 0 10 10'><circle cx='5' cy='5' r
     width="32dp" height="32dp"/>
```

# 9. Debugging

RmlUI ships with a DOM debugger that you can open in-game to inspect elements, check computed styles, and view event logs.

```
-- Enable the debugger for the shared context.
-- Call this AFTER the context has been created.
RmlUi.SetDebugContext('shared')

-- A handy pattern: toggle via a build flag in rml_setup.lua
local DEBUG_RMLUI = true   -- set false before shipping
if DEBUG_RMLUI then
    RmlUi.SetDebugContext('shared')
end
```

Once enabled, a small panel appears in the game window. Click **Debug** to open the inspector or **Log** for the event log.

> ■ **NOTE**
>
> The debugger attaches to one context at a time. Using the shared context means you see all widgets in one place — very useful for tracking cross-widget interactions.

# 10. Best Practices

**■ Use dp units everywhere**

The dp unit scales with context.dp_ratio, so your UI automatically adapts to different resolutions and the player's ui_scale setting.

**■ One shared context per game**

Beyond All Reason and most community projects use a single 'shared' context. Documents in the same context can reference the same data models and are Z-ordered relative to each other.

**■ Keep data models flat when possible**

Top-level writes via dm_handle.key = value are automatically dirty-marked. Deeply nested structures require manual __SetDirty calls — simpler models mean fewer bugs.

■ **Separate per-document and shared styles**

Put styles unique to one document in a <style> block inside the .rml file. Put styles shared across multiple documents in a .rcss file and link it.

■ **Guard against missing RmlUi**

Always start widgets with 'if not RmlUi then return end' to prevent errors in environments where RmlUI is unavailable.

■ **Check return values**

Both OpenDataModel and LoadDocument can fail silently if paths are wrong. Always check for nil and log an error so you know immediately what happened.

■ **Organise by widget folder**

Group luaui/widgets/my_widget/my_widget.lua, .rml, and .rcss together. This makes the project navigable and each widget self-contained.

# 11. RCSS Gotchas

RCSS closely follows CSS2 but has several important differences that will trip up web developers:

• **No default stylesheet**

RmlUI ships with zero default styles. block/inline, heading sizes, list bullets — none of it exists unless you add it. Copy the HTML4 base sheet from the RmlUI docs as a starting point.

• **RGBA alpha is 0–255**

rgba(255, 0, 0, 128) means 50% transparent red. CSS uses 0.0–1.0 for alpha. The opacity property still uses 0.0–1.0.

• **Border shorthand has no style keyword**

border: 1dp #6c7086; works. border: 1dp solid #6c7086; does NOT work. Only solid borders are supported; border-style is unavailable.

• **background-* properties use decorators**

background-color works normally. background-image, background-size, etc. are replaced by the decorator system. See the RmlUI docs for syntax.

• **No list styling**

list-style-type and friends are not supported. ul/ol/li have no special rendering — treat them as plain div-like elements.

• **Input text is set by content, not value**

For and the displayed text comes from the element's text content, not the value attribute (unlike HTML).

• **pointer-events: auto required**

By default elements do not receive mouse events. Add pointer-events: auto to any element that needs to be interactive.

# 12. Differences from Upstream RmlUI

The Recoil implementation is based on the official RmlUI library but adds engine-specific features and changes some defaults:

**■ <texture> element**

A custom element that renders engine-managed textures via Recoil texture reference strings. Not present in upstream.

**■ <svg> inline data**

The src attribute accepts raw SVG markup in addition to file paths. Upstream only supports file paths.

**■ VFS file loader**

All file paths are resolved through Recoil's Virtual File System, so .rml and .rcss files work inside .sdz game archives transparently.

**■ Lua bindings via sol2**

The Lua API is provided by RmlSolLua (derived from LoneBoco/RmlSolLua) rather than the upstream Lua plugin, offering tighter engine integration.

**■ dp_ratio via context**

context.dp_ratio is set from code rather than the system DPI, allowing games to honour the player's ui_scale preference.

**■ NOTE**

For everything not listed above, the upstream RmlUI documentation at **https://mikke89.github.io/RmlUiDoc/** applies directly.

# 13. IDE Setup

Configuring your editor for .rml and .rcss file extensions dramatically improves the authoring experience:

## VS Code

1. Open any .rml file in VS Code.
2. Click the language mode indicator in the status bar (usually shows 'Plain Text').
3. Choose **Configure File Association for '.rml'**.
4. Select **HTML** from the list.

5. Repeat for .rcss files, selecting **CSS**.

### Lua Language Server (lua-ls)

Add the Recoil type annotations to your workspace for autocomplete on `RmlUi`, `Spring`, and all widget APIs. See the Recoil docs guide on the Lua Language Server for details.

# 14. Quick Reference

### Lua API summary

| Call | Description |
|---|---|
| RmlUi.CreateContext(name) | Create a new rendering context |
| RmlUi.GetContext(name) | Retrieve an existing context by name |
| RmlUi.LoadFontFace(path, fallback) | Register a .ttf font (fallback=true recommended) |
| RmlUi.SetMouseCursorAlias(css, engine) | Map a CSS cursor name to an engine cursor name |
| RmlUi.SetDebugContext(name) | Attach the DOM debugger to a named context |
| ctx:OpenDataModel(name, tbl) | Create reactive data model from Lua table; store the return value! |
| ctx:RemoveDataModel(name) | Destroy a data model and free its memory |
| ctx:LoadDocument(path, scope) | Parse an RML file; scope is used for normal (on*) events |
| doc:Show() | Make the document visible |
| doc:Hide() | Hide the document (keeps it in memory) |
| doc:Close() | Destroy the document |
| doc:ReloadStyleSheet() | Reload linked .rcss files (useful during development) |
| dm:__GetTable() | Get the underlying Lua table for iteration |
| dm:__SetDirty(key) | Mark a top-level model key as changed; triggers re-render |
| dm.key = value | Write a top-level value; auto-marks dirty |

### Useful links

• **RmlUI official documentation** — https://mikke89.github.io/RmlUiDoc/

- **RmlUI data bindings reference** — https://mikke89.github.io/RmlUiDoc/pages/data_bindings/views_and_controllers.html
- **RmlUI RCSS reference** — https://mikke89.github.io/RmlUiDoc/pages/rcss.html
- **RmlUI HTML4 base stylesheet** — https://mikke89.github.io/RmlUiDoc/pages/rml/html4_style_sheet.html
- **RmlUI decorators (backgrounds)** — https://mikke89.github.io/RmlUiDoc/pages/rcss/decorators.html
- **RmlUI GitHub** — https://github.com/mikke89/RmlUi
- **RecoilEngine source (Rml/)** — https://github.com/beyond-all-reason/RecoilEngine/tree/master/rts/Rml
- **Recoil texture reference strings** — See engine docs: articles/texture-reference-strings